

Gray-in-Young: A Generational Garbage Collection for Processing-in-Memory

Ryu Morimoto
mryuu@g.ecc.u-tokyo.ac.jp
The University of Tokyo
Japan

Kazuki Ichinose
ichinose-kazuki657@g.ecc.u-
tokyo.ac.jp
The University of Tokyo
Japan

Tomoharu Ugawa
tugawa@acm.org
The University of Tokyo
Japan

Abstract

Processing-in-memory (PIM) is a promising approach to overcome the performance bottleneck caused by the gap between CPU speed and memory speed, known as the memory wall problem. The UPMEM PIM-enabled memory is the first commercialized general-purpose PIM accelerator, to which the program running on the host CPU offloads computation kernels. Its DRAM Processing Units (DPUs) are general-purpose processors and have the flexibility to run various computation kernels. However, there is no support for programming in managed languages with garbage collection (GC). In this paper, we design GC for DPUs, which is a key component of managed runtimes. Our GC is a parallel generational GC, whose young space is in scratch pad memory (SPM). The GC updates pointers in promoting objects before copying them to old space in DRAM to reduce DRAM accesses. It also determines class information needed for minor GC of each computation kernel at compile time and caches them in SPM. The major GC routines are compiled in a separate binary so that the binaries of computation kernels fit in 24 KB of program memory. The evaluation results using a micro benchmark showed that our proposed techniques reduced up to 85.9% DRAM accesses and improved performance by 46.2% for our benchmark. The GC scaled up to 11 threads, and the remaining code size for the GC routine was only 4.3 KB after separating 6.9 KB of major GC.

CCS Concepts: • Software and its engineering → Garbage collection; Object oriented languages.

Keywords: Processing-in-Memory (PIM), Generational GC, Scratch Pad Memory, DMA

ACM Reference Format:

Ryu Morimoto, Kazuki Ichinose, and Tomoharu Ugawa. 2025. Gray-in-Young: A Generational Garbage Collection for Processing-in-Memory. In *Proceedings of the 2025 ACM SIGPLAN International Symposium on Memory Management (ISMM '25)*, June 17, 2025, Seoul,



This work is licensed under a Creative Commons Attribution 4.0 International License.

ISMM '25, Seoul, Republic of Korea

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1610-2/25/06

<https://doi.org/10.1145/3735950.3735961>

Republic of Korea. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3735950.3735961>

1 Introduction

Processing-in-memory (PIM) is attracting attention as a promising approach to overcome the performance bottleneck caused by the gap between CPU speed and memory speed, known as the memory wall problem [30]. In modern computer systems, data is stored in DRAM and moved to the CPU through the memory bus and the cache hierarchy. For memory intensive applications with limited locality, the data in cache is hardly reused, and the memory bus cannot deliver data to the CPU as fast as the CPU can process the data. The PIM approach processes data at the place where the data resides, which can reduce the data transfer overhead.

The UPMEM PIM system is the first commercialized PIM system. It is a computer system equipped with UPMEM PIM-enabled memory as an accelerator. The UPMEM PIM-enabled memory has general-purpose processors called DRAM Processing Units (DPUs) embedded in the DRAM chips. In programming of the UPMEM PIM system, the host CPU executes the main program and offloads computation kernels to the DPUs.

DPUs are sufficiently rich in features to run complicated computation kernels. First, DPUs have an instruction set of general-purpose processors. Second, unlike GPUs, DPUs have multiple hardware threads that run independently and support the SPMD programming model. Third, each DPU has a relatively large amount of DRAM (64 MB per DPU) and data in DRAM is preserved across multiple calls of computation kernels. These features enable various computation kernels to run on DPUs. For example, several in-memory database for UPMEM have been proposed [13, 16]. They store key-value pairs in DRAM of DPUs and process (parts of) queries in the DPUs as computation kernels. Such kernels manipulate trees or graphs, and tend to be complex.

However, support for programming complex computation kernels is limited. Although the UPMEM SDK [27] provides Java and Python APIs for the host programs, programs for DPUs must be written in the C language. In order to mitigate the burden of developing complex computation kernels and reduce the risk of memory bugs, managed languages with garbage collection (GC) for DPUs are desired. Therefore, we

are developing a Java programming framework that offloads kernels to DPUs [14]. In the framework, a garbage collected heap is located in the DRAM of each DPU. The computation kernels are implemented as methods of objects in the DPU's heap. Each kernel method is compiled into a single binary for the DPU. The host program loads the binary and launches the kernel method to offload the computation to the DPUs.

In this paper, we propose GC for DPUs, which is a key component of managed runtimes. There are three challenges for GC for DPUs. The first challenge is that we have to utilize a small scratch pad memory (SPM) attached to each DPU effectively and avoid DRAM accesses as much as possible. DPUs access DRAM through direct memory access (DMA). Thus, DRAM access is much slower than SPM access. SPM can be accessed in a single cycle, while DRAM access takes around $70 + 0.5n$ cycles for n bytes [11]. The second challenge is that GC must be parallel to utilize multiple hardware threads of DPUs. The program in DPU must have at least 11 active threads to fully utilize the processor's pipeline [11]. The third challenge is that we have to reduce the binary size of the computation kernel. A DPU has a separate program memory, and its size is only 24 KB. Every binary will contain GC as well as the computation kernel, so the binary size of the GC routines must be minimized.

To address these challenges, our GC is designed as a generational GC whose young generation space (young space) is located in the SPM while the old generation space (old space) is located in the DRAM. Its minor collection runs in parallel and promotes all the live objects in young space and empties young space. It performs a minor collection when the computation kernel is completed as well as when the young space becomes full.

To reduce DRAM accesses in the minor collection, we introduce two techniques. First, we update the pointers of promoting objects while they are in young space. If we updated pointers after copying the objects to old space, a lot of DRAM accesses at the pointer-size granularity would be required. Second, we cache the object layout information (OLI) in the class table in SPM. Because we empty young space at the end of the computation kernel, the objects in young space are only the ones created by the current computation kernel. Thus, the set of classes of those objects can be statically determined. On the basis of this observation, we create the image of the cache of OLIs when we compile the computation kernel.

To reduce the binary size, only the minor collection routines are linked to the computation kernel. When old space becomes full, the dedicated binary for the major collection is loaded to the DPUs by the host program. In this paper, we mainly focus on the minor collection and leave the optimization of the major collection as future work.

We implemented the proposed GC together with a simple Java programming framework for UPMEM and evaluated the proposed GC using a tree-constructing micro benchmark.

The evaluation results showed that our proposed technique to reduce DRAM access reduces up to 85.9% DRAM accesses and improved performance by 46.2% for our benchmark. The GC scaled up to 11 threads, and the remaining code size for the GC routine was only 4.3 KB after separating 6.9 KB of the routines for the major collection.

2 Background

2.1 UPMEM PIM System

PIM is an approach that processes data in memory chips where the data resides instead of moving the data to the CPU. This improves the performance of memory intensive applications.

The UPMEM PIM system is the first commercialized PIM system[28]. It is a computer system equipped with UPMEM PIM-enabled memory as an accelerator. The UPMEM PIM-enabled memory has general-purpose processors called DRAM Processing Units DPUs embedded in the DRAM chips. In programming of the UPMEM PIM system, the host CPU executes the main program and offloads a computation kernel to the DPUs. Once the DPUs are launched, the DPUs execute the kernel independently of the host CPU.

The UPMEM PIM-enabled memory is a general-purpose PIM accelerator, unlike other domain-specific PIM accelerators [12, 17–19, 21, 22]. DPU has an instruction set of general-purpose processors and supports the SPMD programming model. Furthermore, each DPU has a relatively large amount of DRAM, and data in DRAM is preserved across multiple calls of kernels. Thanks to these features, the UPMEM PIM-enabled memory can run various kernels. For example, in-memory databases may store data in the DRAM of DPUs and run queries on the DPUs as computation kernels [16]. In this example, data in the DPU may be organized as a tree, and the kernel may traverse the tree to find the data.

2.1.1 DPU. A DPU is an in-order 32-bit RISC core with 24 hardware threads[29]. Instructions of the same thread are dispatched at least 11 cycles apart due to the pipeline structure [11]. This means that the application must have at least 11 active threads to fully utilize the pipeline.

A DPU has a 64-KB scratch pad memory (SPM), a 64-MB DRAM, and a 24-KB instruction memory. They are not shared with other DPUs. DPUs access SPM with normal load/store instructions. SPM can be accessed up to 8-byte data every cycle.

In contrast, DPUs access DRAM through direct memory access (DMA), and thus DRAM access is much slower than SPM access. Each DMA operation can transfer from 8 bytes to 2 KB of contiguous data. DRAM access takes around $77 + 0.5n$ cycles for reading n bytes and $61 + 0.5n$ cycles for writing n bytes [11].

To examine if multi-threading mitigates the DRAM access latency, we measured the throughput of DMA at transfer sizes of 8, 16, 64, and 1024 bytes, with the number of threads

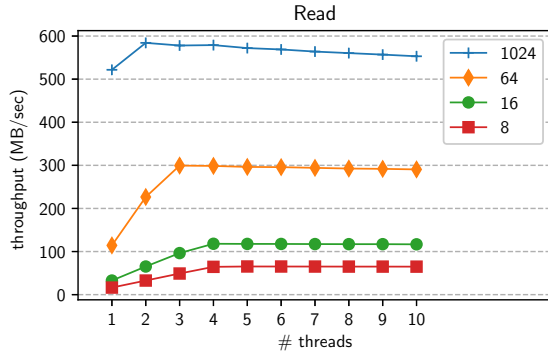


Figure 1. DRAM read throughput; different lines represent different transfer sizes (8, 16, 64, and 1024 bytes)

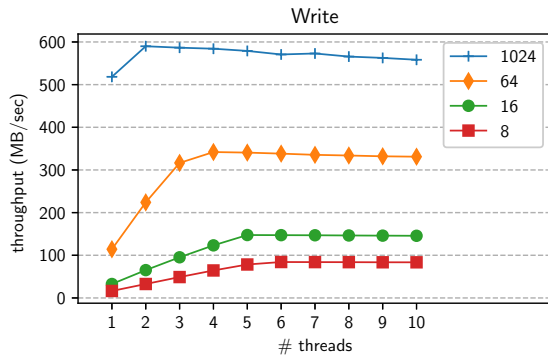


Figure 2. DRAM write throughput; different lines represent different transfer sizes (8, 16, 64, and 1024 bytes)

ranging from 1 to 10. In this experimentation, we transferred 32 KB of data from DRAM to SPM for read, and from SPM to DRAM for write. Data is divided across threads evenly, and each thread transfers the data in parallel using DMA. Figures 1 and 2 plot the throughput against the number of threads. Different lines represent different transfer sizes of DMA. This result shows that, when the transfer size is small, reading/writing with multiple threads improves the throughput. However, no matter how many threads are used, transferring with a large size is faster.

2.1.2 Programming of UPMEM PIM. An application program of the UPMEM PIM system consists of two parts: the host program and the computation kernels. The host program is the main program that can be written in C, C++, Java, or Python. Using APIs provided by the UPMEM SDK, the host program loads the binary program of the kernel to the DPUs and launches the DPUs.

The programs of the kernels are written in C. UPMEM SDK provides an LLVM-based compiler stack and a runtime library for DPUs, which allows general-purpose parallel programming. The kernels are compiled to binary programs in advance.

The application may use multiple kernels, and they can be called multiple times. To call the same kernel multiple times, the host program loads the binary once and launches the DPU multiple times. To call a different kernel, the host program loads a new binary and launches the DPU. When a new binary is loaded, the contents of SPM are cleared. However, the contents of DRAM are always preserved.

2.2 Java Programming Framework for UPMEM

Our GC is designed for a managed runtime for DPUs, which is a part of Java programming framework for UPMEM. To the best of our knowledge, no such frameworks are currently available. However, we believe that high-level programming frameworks for UPMEM will be developed in the future to mitigate the burden of developing complicated computation kernels. We prototyped a managed runtime for the Java programming framework for UPMEM that offloads kernels to DPUs [14]. Although our GC does not target only our framework and the details of our framework are out of the scope of this paper, we describe our framework briefly to make the discussion concrete.

Our framework allows the programmers to write the entire application of the UPMEM PIM system in a single Java program. The framework provides a view of a distributed system to the programmer; the host CPU and the DPUs have their own heaps. The host program can create an object in a DPU's heap and obtain its handle. The objects created in the DPU's heap survive across multiple calls of computation kernels. Remark that we concentrate on GC that manages a heap in a single DPU in this paper; distributed GC to collect cycles through the handles is out of scope of this paper.

The computation kernels are implemented as methods of objects in the DPU's heap. To call a computation kernel, the host program calls the method through the handle. Then, the framework automatically loads the corresponding binary to the DPU and launches it.

When the method running in a DPU creates an object, the object is allocated in the DPU's heap. When the heap becomes full, all the threads in the DPU stop at GC safe-points, and GC is triggered. Note that the methods may be executed with multiple threads using the SPMD-programming support provided by our framework.

The kernel methods are compiled to binaries in advance. Our framework determines methods that can be called from the host program (*entry methods*) on the basis of the programmer's annotations. For each of the entry methods, it then determines the set of methods that can be called from the entry method. Those methods are translated into C and compiled together with the entry method into a single binary for the DPU.

However, if the binary is too large to fit in the 24-KB instruction memory, the framework splits the binary into multiple binaries and executes them using a program overlay technique. Each binary contains some of the methods that

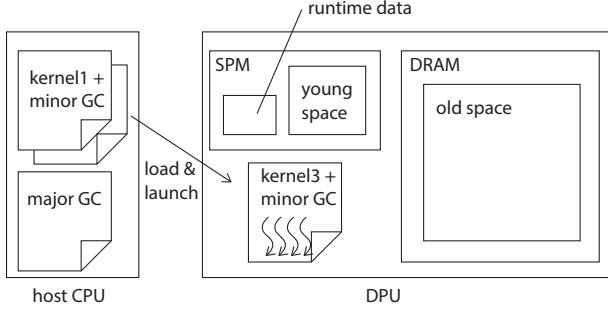


Figure 3. Overview

can be called from the entry method. When a method that is not in the currently running binary is called during the execution in the DPU, the framework saves the current state, such as the Java stack, and loads the binary that contains the callee method.

3 Garbage Collection for DPU

3.1 Design Goals

The design goals of our GC are the following.

- To reduce DRAM accesses as much as possible and access DRAM as large granularity as possible if necessary to avoid DMA overhead.
- To parallelize the GC to utilize the pipeline.
- To reduce the size of the GC routines because GC routines are linked to every computation kernel.

Note that, because DPU does not have cache memory, the order of objects in the heap does not affect the performance.

3.2 Overview

Figure 3 shows the overview of our GC. Our generational GC consists of two generations. Young space is located in SPM and old space is located in DRAM. All small objects are created in young space. This design is expected to improve the mutator's performance because the allocator zeros new objects, and a large amount of writes occur to newly allocated objects [1]. A disadvantage of this design is that write barriers are required to track old-to-young pointers. Moreover, the overhead of write barriers are expected to be larger than standard implementations [31], as we cannot inline the write barrier to keep the binary size small.

When young space becomes full, a minor collection is invoked and all live objects in young space are promoted, i.e., moved to old space; it does not have an aging mechanism. Therefore, after every minor collection, the young space becomes empty. The remembered set, which is located in SPM, is also cleared at this point.

To reduce the binary size, we link only the minor collection routines to the computation kernel. After a minor collection, if old space is almost full, a major collection is invoked. The major collection routines are compiled to a

separate binary. Thus, the host program loads this binary when a major collection is invoked. This is realized by using the program overlay mechanism of the framework that we introduced in Section 2.2.

We perform a minor collection when the kernel finishes as well as when young space becomes full. Furthermore, we also perform a minor collection before the control is returned to the host CPU for swapping the binary using the program overlay mechanism. One of the reasons for this is that SPM is cleared when a new binary is loaded to the DPU. Thus, by performing a minor collection, we move all live objects to DRAM in case the host CPU calls a different kernel. Another reason is that temporary objects created during the execution of the kernel will be dead when the kernel finishes.

The minor collection is performed in parallel, regardless of the number of the mutator threads. In DPU, every thread is tied to a hardware thread, and the number of hardware threads to use is defined when a binary is created. Thus, we prepare a sufficient number of threads for both the mutator and GC. If the mutator requires fewer threads than GC, some of them execute the mutator program while the others wait for GC. When GC is requested, the threads executing the mutator program stop at GC safe-points, and all the threads start GC.

3.3 Minor Collection Algorithm

We designed two algorithms with different strategies for promoting objects. One is a depth-first search (DFS)-based algorithm that updates pointers of promoting objects before they are copied to old space to reduce DRAM access. The other is a compaction-based algorithm that copies live objects to old space en masse to minimize DMA overhead, in addition to updating pointers before copying. Because our preliminary evaluation with a single thread showed that the DFS-based algorithm was better, we parallelized only the DFS-based one.

3.3.1 Problems in Promoting Objects. We depict the problems in promoting objects by using Cheney's copying algorithm [3] as an example. Cheney's algorithm copies live objects from from-space to to-space without using an extra data structure to keep track of the *gray* objects, which are objects that GC has found but not yet scanned in the tricolor abstraction [5]. It uses to-space as a work queue and copies live objects in a breadth-first manner. In the generational GC, from-space is young space in SPM and to-space is old space in DRAM.

Figure 4 illustrates the process of copying objects A and B. It first copies the object A to old space to create A' (Figure 4 (2)). But, at this point, A' has a pointer to the object B in young space. Then, it scans the object A' and finds a pointer to B. As this pointer is pointing to young space, the algorithm copies the object B to old space to create B'. After that, it replaces the pointer in A' to B with a pointer to B' (Figure 4

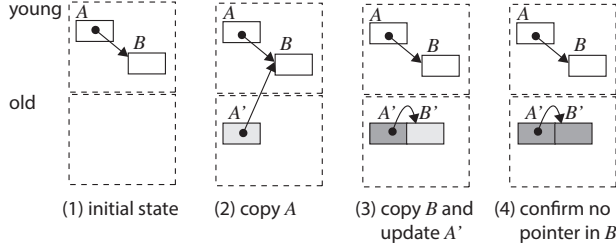


Figure 4. Copy with Cheney's copying algorithm; light gray objects may have pointers to young space, and dark gray objects do not have pointers to young space.

(3)). If A' had more pointers, the algorithm would update them one by one in the same way. After that, the algorithm advances to the next object, B' , and repeats the same process until all the objects in old space are scanned (Figure 4 (4)).

This algorithm may be inefficient in DPU for the following reasons. First, it scans objects in old space to find pointers during traversal because gray objects are in old space. Second, updating pointers of objects is also performed in old space. These scanning and updating operations in old space cause a lot of pointer-sized accesses to DRAM. Third, the algorithm copies objects one-by-one. Typical object sizes are still much smaller than the maximum transfer size of the DMA controller, which is 2048 bytes.

Because of these reasons, a minor collection involves $N + 2NR$ DMA operations for copying objects, where N is the number of live objects in young space and R is the average number of pointers in objects. These potential inefficiencies are not only in Cheney's algorithm. Rather, they are shared with copying algorithms that place gray objects in to-space [6].

3.3.2 Gray-in-Young Algorithm. We propose a DFS-based algorithm, Gray-in-Young (GiY), that updates pointers of promoting objects before they are copied to old space. GiY algorithm keeps gray objects in young space and copies them after they become *black*, which are objects that have been scanned. It traverses the object graph *in young space* and copies reachable objects in preorder. But it does not copy the objects immediately when it visits them. When it visits an object O , it allocates memory for its children's copies. The copies of the children are not filled at this point; they are left as *empty shells*. When the copy of a child is allocated, it updates the pointer in O to the child's copy. Once all the pointers in O are updated, it copies the contents of O to its copy with a single DMA operation.

Figure 5 illustrates the GiY algorithm from the same state of the heap as Figure 4. GiY traverses the object graph from object A . When GiY visits object A , it allocates memory for the copy of A , A' , in old space (Figure 5 (2)). Then, GiY scans A . As a pointer to B is found, it allocates memory for the copy of B , B' , in old space and replaces the pointer in A

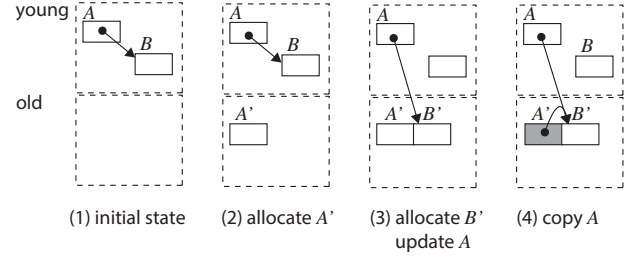


Figure 5. Copy with GiY algorithm; white objects in old space are empty shells.

```

1 def copy(r)
2   // prepare
3   if r.forwarding != null
4     return r.forwarding
5   r.forwarding = alloc()
6   push(stack, r)
7   // DFS traversal
8   while is_not_empty(stack)
9     o = pop(stack)
10    foreach f in Pointers(o)
11      p = o[f]
12      if in_young(p)
13        if p.forwarding == null
14          p.forwarding = alloc()
15          push(stack, p)
16          o[f] = p.forwarding
17    dma_copy(o, o.forwarding, o.size)
18    return r.forwarding

```

Figure 6. Copying function of the GiY algorithm.

B with a pointer to B' . Now, all the pointers in A have been updated (Figure 5 (3)). Then, GiY copies the contents of A to A' with a single DMA operation (Figure 5 (4)). After that, it advances to the object B and repeats the same process.

Figure 6 shows the pseudo code of the copying function of the GiY algorithm. This function promotes the given root object r and its transitive closure in two steps: the prepare step and the DFS traversal step. It first checks if memory for the copy of r has already been allocated. If so, the copying function returns the address of the copy's memory, which is stored in the forwarding field. Note that r 's copy may be an empty shell at this point and filled later. If the forwarding field is null, r has not been visited yet. In this case, the function allocates memory for the copy of r and pushes r to the GC stack. Then, the function traverses the transitive closure of r in a DFS manner using the GC stack.

In the DFS traversal, this function pops the top object o from the GC stack and scans o (Line 10). Here, $\text{Pointers}(o)$ returns the set of pointer fields in the object o . For each pointer p of o , the function allocates the copy of the object pointed by p if it has not been allocated yet. The address of the copy is stored in the forwarding field of p . Then, it replaces the pointer p of o with a pointer to the copy. Finally, when all the pointers in o are updated, the function copies

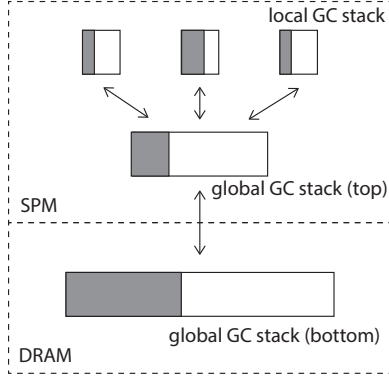


Figure 7. GC stack

the contents of o to its copy with a single DMA operation (Line 17).

Note that we could immediately scan p at Line 15 rather than pushing it to the GC stack. However, we delayed scanning it and pushed p to the GC stack to parallelize the traversal as we describe in Section 3.3.3.

This algorithm performs only a single DMA operation for each object. Thus, the total number of DMA operations in a minor collection is N , where N is the number of live objects in young space.

3.3.3 Parallel GiY Algorithm. The GiY algorithm is parallelized in a straightforward way. For root scanning, the threads that were executing mutator threads scan their Java stacks. The remembered set and the export table are scanned by the primary thread.

During traversal, all the threads share workload through the GC stack. When the GiY algorithm in Figure 6 finds a pointer p in Line 11 during scanning an object o , it allocates memory for the copy of p 's target (Line 14). However, the current thread does not immediately scan p 's target. Rather, the thread pushes p to the GC stack. Later, possibly another thread pops p from the GC stack in Line 9 and scans it.

The GC stack is designed to use limited space in SPM, to encourage load balancing, and to avoid frequent DRAM accesses. As shown in Figure 7, each thread has a local GC stack, which is in SPM. When the local GC stack overflows, the thread moves half of the contents of the local GC stack to the shared GC stack. Local GC stacks are very small (4 entries each) so that the workload in a local GC stack can be frequently exported to the shared GC stack. The shared GC stack consists of two layers; the top layer is in SPM and the bottom layer is in DRAM. When the top layer overflows, half of its contents are moved to the bottom layer. The top layer is sufficiently large (256 entries) so that it does not overflow frequently.

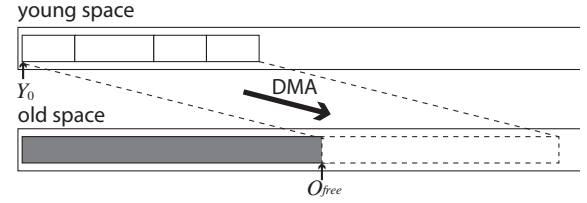


Figure 8. Batch copy of the compaction-based algorithm.

3.3.4 Compaction-Based Algorithm. The second algorithm is a compaction-based algorithm that copies live objects to old space en masse. This algorithm performs compaction in young space and copies live objects to old space using DMA with the maximum transfer size of the DMA controller. Unlike a normal compaction algorithm, this algorithm updates pointers between promoting objects so that they are valid after the objects are copied to old space.

Our algorithm is based on the Lisp2 algorithm [6]. The Lisp2 algorithm is the mark-and-compact algorithm that performs the following three passes over the heap after marking live objects.

1. First pass computes the addresses to which live objects will be moved. The computed address is stored in the forwarding field in the object header.
2. Second pass updates pointers in live objects so that they point to the new addresses of their targets.
3. Third pass moves live objects to their new addresses.

After compaction, all the live objects in young space are gathered in a contiguous area at the beginning of young space (Figure 8). Our algorithm copies those live objects to old space using DMA with the maximum transfer size of the DMA controller, which is 2048 bytes. We assume that old space supports bump-pointer allocation so that a contiguous area to which all the live objects are copied can be found.

In the second pass of the Lisp2 algorithm, our algorithm updates pointers so that they point to the addresses to which their targets will be eventually moved, which is in old space. These addresses can be computed in the following expression:

$$\text{new_address} = \text{forwarding} - Y_0 + O_{\text{free}},$$

where Y_0 is the start address of young space, O_{free} is the first address of the free area in old space, and forwarding is the address stored in the forwarding field in the first pass.

This algorithm could be parallelized by using a parallel Lisp2 algorithm, such as Flood et al. [7]. However, we did not implement the parallel version because the GiY algorithm was better in speed and binary size in our preliminary evaluation.

3.4 Caching Object Layout Information

3.4.1 Object Layout Information. When GC scans an object in Line 10 of Figure 6, it refers to the object layout

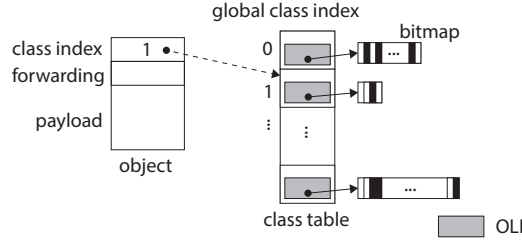


Figure 9. class table; dashed line indicates where the index refers to.

information of the object to find the pointers. We call the data structure that contains this information the *Object Layout Information* (OLI). Because OLIs are in the class table, which is in DRAM, referring to OLIs involves DRAM accesses. To reduce DRAM accesses, we cache the OLIs that may be referenced by the minor collection routine of the current computation kernel in SPM.

Because the number of classes used in an application may be large, the class table is stored in DRAM. The class table is an array of class table entries, each of which contains OLI as shown in Figure 9. OLI consists of an integer indicating the object size and a bitmap indicating the offset of pointer fields in the object. Because the size of the bitmap depends on the number of fields in the object, the size of the bitmap is not bounded in general. Thus, the bitmap is allocated separately from the OLI, and the pointer to the bitmap is stored in the OLI. Without our OLI cache mechanism, every object would have an index of the class table in its header. We call this index a *global class index* because it is unique in the application.

However, the set of OLIs that may be referenced by the minor collection routine of the current computation kernel is limited and can be statically determined when the computation kernel is compiled. Because we invoke a minor collection at the end of every computation kernel, and our minor collection empties young space, young space is empty when the computation kernel is called. Thus, the objects in young space are limited to the ones created by the current computation kernel, and their classes are limited to the ones that explicitly appear in the new expressions in the computation kernel. The minor collection routine scans the objects in young space, whose classes are limited as we described above. Note that we do not support dynamic class loading.

3.4.2 Static Cache of OLIs. Based on this observation, we cache the OLIs that may be referenced by the minor collection routine of the current computation kernel in SPM. We create the image of this cache when we compile the computation kernel, and the image is loaded to SPM when the computation kernel is called. Thus, the cache is static and does not change during the execution of the computation

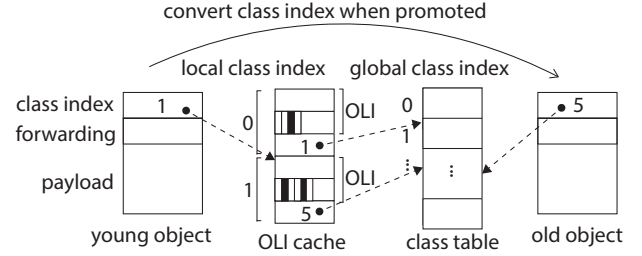


Figure 10. OLI cache; dashed line indicates where the index refers to.

kernel. The images of the cache of all the computation kernels are stored in DRAM, and the initialization routine of each computation kernel loads the corresponding image to SPM.

The cache is an array of the pairs of an OLI and a global class index as shown in Figure 10. The index of the cache is called a *local class index*. The new expressions in the computation kernel are compiled so that the created objects have local class indexes in their headers instead of global ones. Thus, all objects in young space have local class indexes in their headers. When the minor collection scans an object, it refers to the cache with the local class index in the object header to find the OLI for the object.

Objects in old space have global class indexes in their headers because they may be used by different computation kernels. To preserve this invariant, we replace the local class index in the object header with the global one when the object is promoted. More precisely, we replace the index immediately before copying the object to old space in the GiY algorithm in Line 17 of Figure 6.

This design has another advantage. In our design, the size of the bitmap in the cache is relatively tightly bounded because the set of OLIs is limited. Thus, the bitmap can be embedded in the array of OLIs in the cache.

4 Evaluation

We developed the proposed GC together with the prototype of the Java programming framework and experimentally evaluated it in the following aspects.

- Overall performance of the GC, including mutator's throughput.
- Effectiveness of the techniques to reduce DRAM accesses proposed in Sections 3.3.2 and 3.4.
- Scalability of the parallel minor collection proposed in Section 3.3.3.
- Binary size.

4.1 Benchmark Program

Because there are no standard benchmarks for DPUs written in Java, we developed a micro benchmark. Although

Table 1. Average promotion ratio and the number of major and minor collections.

α	7	15	100
average promotion ratio	3.6%	7.6%	50.2%
# minor collection	1508	1508	1510
# major collection	0	0	2

there is a benchmark suits for DPUs [11], they are not suitable for benchmarking GC because they do not frequently creates objects. Our benchmark program imitates the behavior of one DPU in in-memory databases, where data is distributed across DPUs and queries are processed in each DPU. As workload is tend to be skewed, some parts of data are frequently updated while the others are stable.

This program creates a certain size of binary search tree repeatedly. The tree consists of the `InternalNode` and `LeafNode` classes. The type of key is `int` and the values are single-field objects.

The size of the binary tree is a controllable parameter, α . The root of the tree being constructed is pointed to by a local variable. Every time α elements are inserted to the tree, the tree is replaced with an empty tree. Table 1 shows the average promotion ratio and the number of major and minor collections that took place for each α . These numbers are the same for all generational GCs we compared in this section. The numbers of minor collections are similar for every α because this program inserts the same number of elements in total regardless of α . For $\alpha = 100$, two extra minor collections took place because unnecessary minor collections occurred before leaving the major collection's binary.

To simulate the situation where some parts of data is stable, we placed a certain size of data in old space. More precisely, we created a tree whose size is around 1 MB in total before the benchmark starts and kept it alive during the benchmark.

This program fitted in the 24-KB instruction memory of DPUs. Thus, the control was not returned to the host program during the benchmark, except for major collections. This mutator program ran in a single thread.

4.2 Experimental Setup

In this experimentation, we set the size of young space to 28 KB and the size of old space to 10 MB minus 28 KB. When we compare with non-generational GC, we set the size of the heap for the non-generational GC to 10 MB. We could use a larger old space, but we use a small old space to complete benchmarking in a reasonable time while triggering major collections.

DPU programs are compiled using the Clang compiler in the UPMEM SDK version 2023.2.0. The programs are compiled with the `-Oz` optimization level, which produces the

smallest binary. Programs are executed in a single DPU running at 350 MHz, which is embedded in the UPMEM PIM-enabled memory.

To measure the execution time for minor collections, we used the DPU's performance counter, which provides the elapsed clock cycles, and converted it to the elapsed time on the basis of the clock frequency of 350 MHz. On the other hand, we measured the total execution time of the benchmark program using `System.nanoTime()` in the host program. The major collection's time was also measured in the host program. The major collection's time includes the times to load the binary for the major collection to the DPU, to launch the DPU, execution of the major collection, and to load the binary for the benchmark program that invoked the major collection; the time to suspend and resume the computation kernel is not included. The time for the mutator is computed by subtracting the times for minor and major collections from the total execution time.

For all the generational GCs we compared, the major collection is a single-thread Lisp2 mark-compact GC. Development of an efficient major collection for DPUs is left as future work.

4.3 Overall Performance

Figure 11 shows the total elapsed time of the benchmark program for various GCs. They are generational GCs, except for MC.

- DFS has the baseline minor collection that copies objects in DFS-order while scanning objects and updating pointers in DRAM.
- GiY has the GiY minor collection proposed in Section 3.3.2.
- YC has the compaction-based minor collection proposed in Section 3.3.4.
- MC is a non-generational Lisp2 mark-compact GC.

In figure 11, we executed minor collections as well as major collections in a single thread. The top white part of the bars (very small for $\alpha = 7$ and 15) indicates the time for the minor collection. The black part of the bars indicates the time for the major collection or the non-generational GC. The bottom part of the bars indicates the time for the mutator.

This shows that, for every α , total GC times for generational GCs were shorter than that of MC. This showed the effectiveness of the generational GC where young space is located in SPM. Furthermore, the mutator's times for generational GCs were also shorter than that of MC despite the overhead for the generational GC's write barriers. This is because the objects were created in SPM and initialized quickly. Mutator times differed among generational GCs for the same α ; we could not find the reason for these differences. As α increased, the mutator time increased because the insertion of an element to a larger tree took longer time.

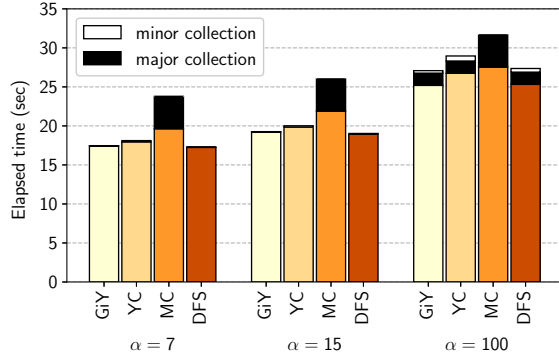


Figure 11. Execution time of the benchmark program.

In this experimentation, the fraction of the minor collection time to the total execution time was small. This is because the mutator and the major collection were slow; they are single-threaded. Multi-threaded programs would create objects more rapidly and invoke minor collections more frequently.

Time for major collections includes the time for loading the binary for the major collection. However, the time for loading the binary was short; 11 milliseconds per major collection. This is only 1.4% of the time for a major collection.

In the rest of this section, we focus on the minor collection. Figure 12 shows the total time for the minor collection in the same experimentation as figure 11. These results show that GiY was faster than the baseline, DFS, but YC was slower than DFS. This implies that compaction was too expensive compared to the benefit of reducing per-object DRAM accesses. As we show in Section 4.4, around 300 objects were promoted in a single minor collection for $\alpha = 100$, for example. Because it takes $61 + 0.5n$ cycles to transfer n bytes to DRAM using DMA, the penalty for per-object DMA in a single collection was $300 \times 61 = 18300$ cycles. In the entire application, the penalty was $18300 \times 1508 = 20810400$ cycles, which is around 59 ms. This is equivalent to 18.5% of the minor collection time for GiY. However, compaction took a longer time than this penalty.

4.4 Effectiveness of Techniques to Reduce DRAM Accesses

To examine the effectiveness of the proposed techniques to reduce DRAM accesses, we compared GiY with its variants. More precisely, we compared the following GCs.

- +U+C is GiY, which updates pointers in SPM, with OLI cache enabled.
- +U-C is GiY that does not cache OLIs.
- -U-C is DFS in Section 4.3. It traverses and updates pointers in DRAM and does not cache OLIs.

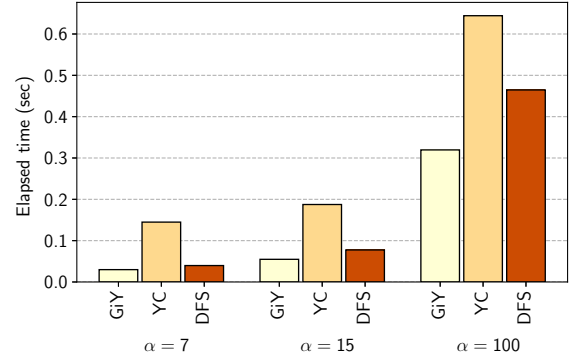


Figure 12. Total minor collection time.

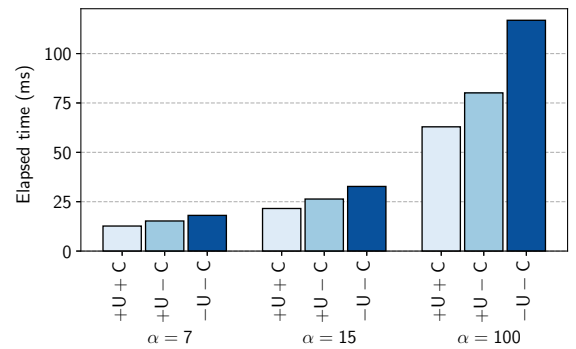


Figure 13. Total minor collection time for the variations of GiY executed in 11 threads.

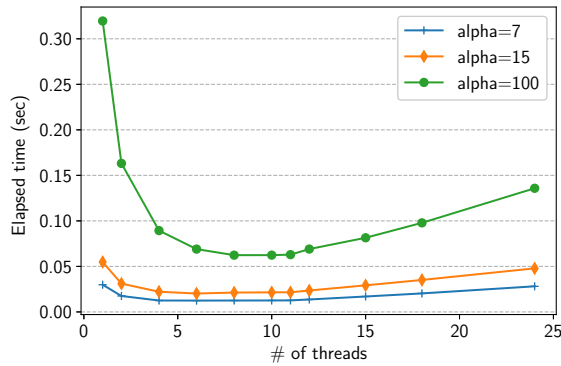
Minor collections for these GCs were executed in 11 threads in this experimentation. For this benchmark program, 64 bytes of SPM was used for OLI cache.

Table 2 shows the number of DRAM accesses and the number of bytes transferred to DRAM for each minor collection, on average. The rows “- copyobj” show those numbers that do not include DRAM accesses for copying objects to old space. Even for +U+C, those numbers were not zero because some of the roots, including old-to-young pointers, were in DRAM. Figure 13 shows the total time for minor collections for these GCs.

These results show that both updating pointers in SPM and caching OLIs were effective. For $\alpha = 100$, the number of DRAM accesses for -U-C was reduced by 36.3% by OLI cache and by 85.9% when both techniques were enabled. Because of these reductions, the total times for minor collections for +U-C and +U+C were 31.5% and 46.2% shorter than that for -U-C, respectively. For smaller α , the effectiveness of the techniques was still significant but not as much as for $\alpha = 100$. This is because, for smaller α , there were fewer objects promoted in each minor collection.

Table 2. DRAM access count and bytes for the variants of GiY.

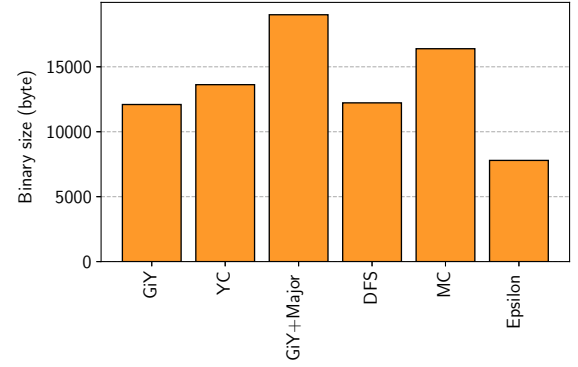
α	GC	+U+C	+U-C	-U-C
7	count	63.3	148.5	207.7
	– copyobj	42.0	127.2	186.4
	bytes	1356.8	2038.0	2511.9
	– copyobj	336.3	1017.5	1491.3
15	count	74.3	254.9	383.5
	– copyobj	29.1	209.8	338.3
	bytes	2398.9	3844.1	4872.6
	– copyobj	233.1	1678.2	2706.7
100	count	341.9	1540.4	2418.6
	– copyobj	42.3	1240.8	2119.0
	bytes	14720.2	24308.1	31333.8
	– copyobj	338.3	9926.4	16952.0

**Figure 14.** Scalability of parallel GiY.

4.5 Scalability of Parallel GiY

To examine the scalability of the parallel GiY, we plotted the total times for minor collections for GiY against the number of threads. Note that the numbers of minor collections that took place were the same regardless of the number of threads. Figure 14 shows the results for $\alpha = 7, 15$, and 100.

This result shows that the throughput was maximized at around 8 threads, and the throughput was 5.12 $\times$ higher than that for a single thread. For $\alpha = 100$, the throughput was the best at 8 threads, but was not degraded much until 11 threads. For smaller α , the throughput was saturated at fewer threads. This is because the number of objects promoted was smaller and there was little workload to be shared among many threads. However, the throughput was not degraded by increasing the number of threads until 11 threads. Therefore, we conclude that parallel GiY scales up to 11 threads.

**Figure 15.** Binary size.

4.6 Binary Sizes

We compared the binary sizes of the computation kernels that contain the minor collection routines. Figure 15 shows the results. In addition to GCs we compared in Section 4.3, we compared

- GiY+Major; GiY plus the major collection routines and
- Epsilon; no GC routines.

The computation kernel we used was almost empty; it contains only a single new instruction so that the allocator and GC are linked.

This result shows that the minor collection routines for GiY, which is the difference between GiY and Epsilon, were around 4.3 KB. If we include the major collection routines, the size of GC routines was around 11.2 KB. This shows that separating the major collection routines is worth doing despite the overhead for the program overlay.

In comparison with GiY and YC, the size of the minor collection routines for YC was larger by 1.5 KB. This shows that the minor collection should be simple to reduce the binary size.

5 Related Work

We proposed GC for DPUs, which has different types of memory: SPM and DRAM. Generational GC is a standard approach in such systems. Lebsack and Chang [20] proposed to place young space in SPM in a hybrid system where both SPM and cache memory are equipped. By placing young space in SPM, they reduced writebacks from the cache to the main memory. Chong et al. [4] placed not only young objects but also frequently accessed objects in SPM. Their work was also done in a hybrid system with SPM and cache memory. In contrast, DPU does not have cache memory and the DRAM is accessed though DMA. Tsai et al. [26] proposed hardware that manages SPM and the main memory, recognizing objects. It automatically moves objects to SPM when they are accessed. In contrast, we do not move old objects to SPM. Relocating frequently accessed old object to

SPM is one of our future work. Therefore, we had to optimize DMA transfer. Akram et al. [1] proposed generational GC for a hybrid system with DRAM and Non-Volatile Memory (NVM). Their motivation was to reduce the writes to NVM to extend its lifetime. On the basis of the observation that new objects tend to be written frequently [1, 25, 32], they placed young space in DRAM. Furthermore, they identified frequently written objects and placed them in DRAM.

Inoue et al. [15] proposed GC for embedded systems with a small amount of SRAM and a large amount of NOR flash memory. NOR flash memory can be read at a byte granularity but can be written at a page granularity. Their memory management is a combination of GC and paging. They used pages in SRAM as cache and wrote back to flash memory if a new page is needed. However, they tried to avoid writebacks by gathering frequently written objects in SRAM (objects in flash memory needed to be moved to SRAM before writing) and preferred GC to paging. We did not take this approach because their approach needs software virtual memory, which is costly in terms of speed and binary size.

Methods for scanning objects are well-studied. Garner et al. [10] surveyed the methods for scanning objects in GC. Our GC simply refers to the OLIs in the class table, but makes a static cache in SPM. Sansom [24] generated code for scanning objects statically for every object layout. This approach can avoid the overhead of referring to the bitmap in OLIs, but specialized routines for object scanning of various object layouts would increase the binary size. Gagnon [8] proposed the *bidirectional object layout* where pointer fields and non-pointer fields are placed on opposite sides of the object header. It is implemented in the SableVM [9]. This layout does not need to refer to the bitmap in OLIs. However, we used a standard object layout to keep GC simple.

Several programming frameworks for the UPMEM PIM system are proposed. UPMEM SDK [27] provides Java and Python APIs for the host programs and an LLVM-based C compiler for DPUs. Its runtime library provides malloc-style memory allocators to manage SPM. However, it does not manage DRAM. Simple PIM [2] provides a high-level abstraction used in distributed systems. DaPPa [23] allows data parallel programming using parallel patterns. DaPPa manages the data movement between the host CPU and DPUs automatically. However, both of them require the programmers to write the computation kernels in C or C-like DSL. They do not provide GC for DPUs.

6 Conclusion and Future Work

We designed a parallel generational GC for DPUs. Our key techniques and design decisions are (1) update pointers of promoting objects while they are in young space, (2) cache

object layout information in SPM, (3) parallelize minor collection, and (4) separate major collection routines from computation kernels. Our evaluation results showed that generational GC that places young space in SPM was better than a non-generational mark-compact GC. The proposed techniques reduced up to 85.9% DRAM accesses and improved performance by 46.2% for our benchmark. Our minor collection scaled up to 11 threads, and the code size for the GC routine was only 4.3 KB after separating 6.9 KB of major GC. Therefore, we conclude that this design is effective for GC for DPUs.

We also designed a compaction-based GC for DPUs. Its design goal is to minimize the overhead of DMA transfer. However, this GC was slow because of the overhead of compaction, and the binary size was large. We found that, in DPU, we have to be careful about trading off more computation for less memory access.

One of our future work is to optimize the major collection, especially parallelization of it. For latency-sensitive applications, incremental major collection may also be a promising approach. In our current implementation, all old objects are in DRAM. Exploring the design space for object placements, including caching old objects in SPM, is also our future work.

Acknowledgments

Parts of this work are supported by JSPS through JSPS KAKENHI grant number JP23K24822.

References

- [1] Shoaib Akram, Jennifer B. Sartor, Kathryn S. McKinley, and Lieven Eeckhout. 2018. Write-ratoning garbage collection for hybrid memories. *ACM SIGPLAN Notices* 53, 4 (June 2018), 62–77. <https://doi.org/10.1145/3296979.3192392>
- [2] Jinfan Chen, Juan Gómez-Luna, Izzat El Hajj, Yuxin Guo, and Onur Mutlu. 2023. *SimplePIM: A Software Framework for Productive and Efficient Processing-in-Memory*. Technical Report. Cornell University Library, arXiv.org. <https://doi.org/10.48550/arxiv.2310.01893>
- [3] C. J. Cheney. 1970. A nonrecursive list compacting algorithm. *Commun. ACM* 13, 11 (Nov. 1970), 677–678. <https://doi.org/10.1145/362790.362798>
- [4] K. F. Chong, C. Y. Ho, and Anthony S. Fong. 2007. Pretenuing in Java by Object Lifetime and Reference Density Using Scratch-Pad Memory. In *Proceedings of 15th EUROMICRO International Conference on Parallel, Distributed and Network-Based Processing*. 205–212. <https://doi.org/10.1109/PDP.2007.67>
- [5] Edsger W. Dijkstra, Leslie Lamport, A. J. Martin, C. S. Scholten, and E. F. M. Steffens. 1978. On-the-fly garbage collection: an exercise in cooperation. *Commun. ACM* 21, 11 (Nov. 1978), 966–975. <https://doi.org/10.1145/359642.359655>
- [6] Robert R. Fenichel and Jerome C. Yochelson. 1969. A LISP garbage-collector for virtual-memory computer systems. *Commun. ACM* 12, 11 (Nov. 1969), 611–612. <https://doi.org/10.1145/363269.363280>
- [7] Christine H. Flood, David Detlefs, Nir Shavit, and Xiaolan Zhang. 2001. Parallel garbage collection for shared memory multiprocessors. In *Proceedings of the 2001 Symposium on JavaTM Virtual Machine Research and Technology Symposium - Volume 1*. USENIX Association, USA.

- [8] Etienne Gagnon. 2003. *A portable research framework for the execution of java bytecode*. Ph.D. Dissertation. McGill University.
- [9] Etienne M. Gagnon and Laurie J. Hendren. 2001. SableVM: a research framework for the efficient execution of java bytecode. In *Proceedings of the 1st Java Virtual Machine Research and Technology Symposium*. USENIX Association, USA, 27–40.
- [10] Robin J. Garner, Stephen M. Blackburn, and Daniel Frampton. 2011. A comprehensive evaluation of object scanning techniques. In *Proceedings of the International Symposium on Memory Management*. ACM, New York, NY, USA, 33–42. <https://doi.org/10.1145/1993478.1993484>
- [11] Juan Gómez-Luna, Izzat El Hajj, Ivan Fernandez, Christina Giannoula, Geraldo Oliveira, and Onur Mutlu. 2021. *Benchmarking a New Paradigm: An Experimental Analysis of a Real Processing-in-Memory Architecture*. Technical Report. Cornell University Library, arXiv.org. <https://doi.org/10.48550/arxiv.2105.03814>
- [12] Mingxuan He, Choungki Song, Ilkon Kim, Chunseok Jeong, Seho Kim, Il Park, Mithuna Thottethodi, and T. N. Vijaykumar. 2020. Newton: A DRAM-maker's Accelerator-in-Memory (AiM) Architecture for Machine Learning. In *Proceedings of the 53rd Annual IEEE/ACM International Symposium on Microarchitecture*. 372–385. <https://doi.org/10.1109/MICRO50266.2020.00040>
- [13] Yifan Hua, Shengan Zheng, Weihang Kong, Cong Zhou, Kaixin Huang, Ruoyan Ma, and Linpeng Huang. 2024. RADAR: A Skew-Resistant and Hotness-Aware Ordered Index Design for Processing-in-Memory Systems. *IEEE Trans. Parallel Distrib. Syst.* 35, 9 (Sept. 2024), 1598–1614. <https://doi.org/10.1109/TPDS.2024.3424853>
- [14] Kazuki Ichinose, Shigeyuki Sato, and Tomoharu Ugawa. 2025. Towards a Java Virtual Machine for Processing-in-Memory (to appear). In *9th International Workshop on Modern Language Runtimes, Ecosystems, and VMs*.
- [15] Akira Inoue, Tomoharu Ugawa, and Shigeru Chiba. 2024. A Managed Memory System for Micro Controllers with NOR Flash Memory. In *Proceedings of the 2024 ACM SIGPLAN International Symposium on Memory Management*. ACM, USA, 57–67. <https://doi.org/10.1145/3652024.3665511>
- [16] Hongbo Kang, Yiwei Zhao, Guy E. Blelloch, Laxman Dhulipala, Yan Gu, Charles McGuffey, and Phillip B. Gibbons. 2022. PIM-Tree: A Skew-Resistant Index for Processing-in-Memory. In *Proceedings of the VLDB Endow.* 16, 4 (Dec. 2022), 946–958. <https://doi.org/10.14778/3574245.3574275>
- [17] Liu Ke et al. 2022. Near-Memory Processing in Action: Accelerating Personalized Recommendation With AxDIMM. *IEEE Micro* 42, 1 (2022), 116–127. <https://doi.org/10.1109/MM.2021.3097700>
- [18] Jin Hyun Kim et al. 2022. Aquabolt-XL HBM2-PIM, LPDDR5-PIM With In-Memory Processing, and AXDIMM With Acceleration Buffer. *IEEE Micro* 42, 3 (2022), 20–30. <https://doi.org/10.1109/MM.2022.3164651>
- [19] Young-Cheon Kwon et al. 2021. 25.4 A 20 nm 6 GB Function-In-Memory DRAM, Based on HBM2 with a 1.2 TFLOPS Programmable Computing Unit Using Bank-Level Parallelism, for Machine Learning Applications. In *Proceedings of the 2021 IEEE International Solid-State Circuits Conference*, Vol. 64. 350–352. <https://doi.org/10.1109/ISSCC42613.2021.9365862>
- [20] C.S. Lebsack and J.M. Chang. 2005. Using scratchpad to exploit object locality in Java. In *Proceedings of the 2005 International Conference on Computer Design*. 381–386. <https://doi.org/10.1109/ICCD.2005.111>
- [21] Sukhan Lee et al. 2021. Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology : Industrial Product. In *Proceedings of the 2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture*. 43–56. <https://doi.org/10.1109/ISCA52012.2021.00013>
- [22] Seongju Lee et al. 2022. A 1 ynm 1.25 V 8 Gb, 16 Gb/s/pin GDDR6-based Accelerator-in-Memory supporting 1 TFLOPS MAC Operation and Various Activation Functions for Deep-Learning Applications. In *Proceedings of the 2022 IEEE International Solid-State Circuits Conference*, Vol. 65. 1–3. <https://doi.org/10.1109/ISSCC42614.2022.9731711>
- [23] Geraldo F. Oliveira, Alain Kohli, David Novo, Juan Gómez-Luna, and Onur Mutlu. 2023. DaPPA: A Data-Parallel Framework for Processing-in-Memory Architectures, arXiv.org. <https://arxiv.org/abs/2310.10168>
- [24] Patrick M Sansom. 1991. Dual-mode garbage collection. In *Proceedings of the Workshop on the Parallel Implementation of Functional Languages*, Vol. 12. 283–310.
- [25] Jennifer B. Sartor, Wim Heirman, Stephen M. Blackburn, Lieven Eeckhout, and Kathryn S. McKinley. 2014. Cooperative cache scrubbing. In *Proceedings of the 23rd International Conference on Parallel Architectures and Compilation*. New York, NY, USA, 15–26. <https://doi.org/10.1145/2628071.2628083>
- [26] Po-An Tsai, Yee Ling Gan, and Daniel Sanchez. 2018. Rethinking the Memory Hierarchy for Modern Languages. In *Proceedings of the 51st Annual IEEE/ACM International Symposium on Microarchitecture*. 203–216. <https://doi.org/10.1109/MICRO.2018.00025>
- [27] UPMEM. 2024. Software development kit (sdk). <https://sdk.upmem.com/> <https://sdk.upmem.com/>
- [28] UPMEM. 2024. UPMEM. <https://www.upmem.com/> <https://www.upmem.com/technology/>
- [29] UPMEM. 2024. User Manual. <https://sdk.upmem.com/2023.2.0/index.html> <https://sdk.upmem.com/2023.2.0/index.html>
- [30] Wm. A. Wulf and Sally A. McKee. 1995. Hitting the memory wall: implications of the obvious. *SIGARCH Comput. Archit. News* 23, 1 (March 1995), 20–24. <https://doi.org/10.1145/216585.216588>
- [31] Karen Zee and Martin Rinard. 2002. Write barrier removal by static analysis. *ACM SIGPLAN Notices* 37, 11 (Nov. 2002), 191–210. <https://doi.org/10.1145/583854.582439>
- [32] Yi Zhao, Jin Shi, Kai Zheng, Haichuan Wang, Haibo Lin, and Ling Shao. 2009. Allocation wall: a limiting factor of Java applications on emerging multi-core platforms. *ACM SIGPLAN Notices* 44, 10 (Oct. 2009), 361–376. <https://doi.org/10.1145/1639949.1640116>

Received 2025-03-18; accepted 2025-05-03